

```
{camera:'true'});
face_tracker.on("track", handle_faces);
```

The function *handle_faces* is a callback fired for handling detected regions. As mentioned earlier, *tracking.js* returns a list containing Rect objects. In the application discussed, the detected faces will be tagged via a JavaScript prompt. Once the prompt value is taken, the face is identified and tracked with the given name as well as indexed for UI purposes. The complete code can be obtained at [/githublink](#). If the face is detected initially, or there is a state change of tracking (stop/start), the prompt is re-called and the data is stored within an array. For tracking purposes, each newly obtained Rect object is compared with the previously recorded nearest face. Comparison is based on the minimum Euclidean distance. If not returned, then it is recalculated.

Features extraction and matching

In simple terms, any significant discernible parts of the image can be defined as a feature. These can be corner points, edges or even a group of vectors oriented independently. The process of extracting such information is called feature extraction. Various implementations exist for feature extraction and descriptors, such as SIFT, SURF (feature descriptors) and FAST (corner detection). *Tracking.js* implements BRIEF (Binary Robust Independent Elementary Features) and FAST (Features from Accelerated Segmentation Test). Input to the system is first a gray image. The following code extracts corner points (points of interest) based on FAST.

```
var gray = tracking.Image.grayscale(input_image, width,
height);
var corners = tracking.Fast.findCorners(gray, width, height);
```

Each feature point can be referred to as a location. But to be able to perform any operations, these locations are converted into descriptors, which can be considered as a list of vectors that define a given feature. Comparison operators are applied upon these vectors. To find descriptors, *tracking.js* uses the BRIEF framework to extrapolate descriptor vectors from given feature points.

```
var descriptors = tracking.Brief.getDescriptors(gray, width,
corners);
```

Having got the points of interest from an image as well as their descriptors, we can design a scenario wherein one can track based on templates. Given a video frame and a fixed image, features can be used to match and identify where the fixed image can be located. However, there can be false positives.

```
var matches = tracking.Brief.reciprocalMatch(corner_scene,
descriptor_scene, corner_target, descriptor_target);
// calculates the matching points between the scene and the
target image.
```

```
matches.sort(function(a, b){
    //matches can be further filtered by using a sorting
function
    // Either sort according to number of matches found:
    return b.length - a.length;
    // or sort according to confidence value:
    return b.confidence - a.confidence
})
```

The matches obtained can be sorted on the basis of their length, i.e., the number of matches obtained, and on their confidence value, as to how well the points match. Having arranged the matches, efficient matching of the target template image and the scene image can be carried out. It is simply a task of graphics now. Just iterate over the two arrays and mark the appropriate feature points on the canvas, as follows:

```
function plot_matches(matches){
    for (var i = 0; i < matches.length; i++) {
        var color = "red";
        context.lineWidth = "2px";
        context.fillStyle = color;
        context.strokeStyle = color;

        context.beginPath();
        context.arc(matches[i].keypoint1[0], matches[i].
keypoint1[1], 4, 0, 2*Math.PI);
        context.stroke();
    }
}
```

The above function plots the matches only for the scene image, since the reference context is made with respect to one canvas element. For plotting matches on the target template image, a context reference has to be made to its respective canvas element.

Computer vision on the front-end can be used for various applications, not only to produce various image effects but also applications like browser based gesture control, etc. The advent of JavaScript libraries has helped to make this possible. The code can be found at https://www.github.com/kushalvyas/trackingjs_ofy 

References

- [1] Sebastian Thrun. Udacity lecture on artificial intelligence.
- [2] www.trackingjs.com/
- [3] <https://developer.mozilla.org/en-US/docs/Web/API/Navigator/getUserMedia>
- [4] <https://github.com/eduardolundgren/tracking.js>
- [5] https://github.com/kushalvyas/trackingjs_ofy/

By: Kushal Vyas

The author is an open source enthusiast. His projects involve image searching and recognition, visual tracking and 3D reconstruction. He has worked with many open source frameworks such as OpenCV, tracking.js, Boost libraries, Django, etc. He blogs at www.bitsmakemecrazy.com.